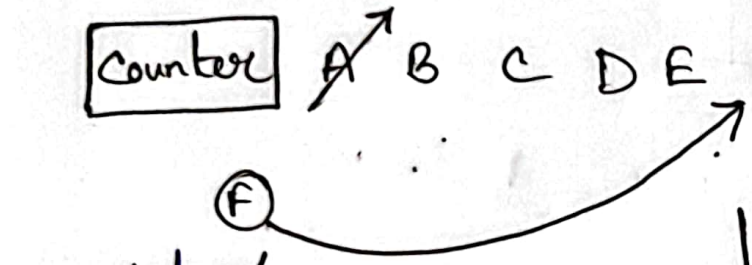
10-07-23

QUEUE

- FIFO → First In First Out



- deletion/removal → 1st value
- Insertion → last

Queue
- → Dequeue
- → Enqueue(elem)
- → Peek

(*) In Queue can use only these operations

⑭ VZF# P?Q##?

→ Dequeue

? → peek

Alphabet → Enqueue

char	Queue Operation	Queue	Return
V	Enqueue	V	
Z	"	VZ	
F	"	VZF	
#	Dequeue	ZF	V
P	Enqueue	ZFP	
?	peek	ZFP	Z
Q	Enqueue	ZFPQ	
#	Dequeue	FPQ	Z

Salicid™

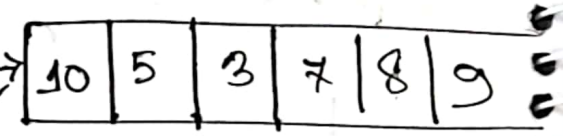
CamScanner

Use Case

#driver code

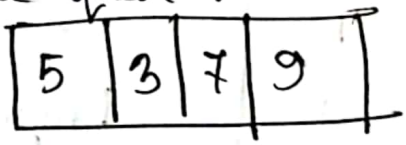
```
given_queue = Queue()
given_queue.enq(10)
given_queue.enq(5)
```

Given Queue:



Given a queue, remove all the even element from the queue.

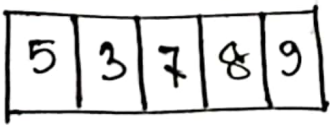
output :



Now,

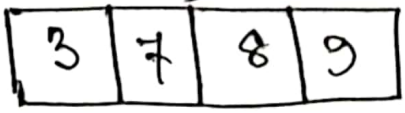
Deq

10



Deq : 5

Q.q.enq(5)

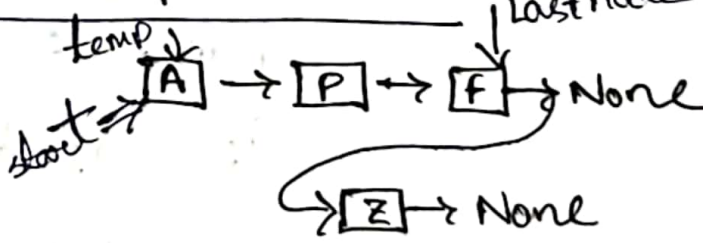


Code:

```
def remove_even(q):
    o_q = Queue()
    while q.peek() != None:
        e = q.dequeue()
        if e % 2 != 0:
            o_q.enqueue(e)
```

Queue Underflow:

Linked List based Queue Implementation:




```
class Node:

class Queue:
    def __init__(self):
        self.start = Node(None, None, None)
    def enqueue(self, elem):
        new_node = Node(elem, None)
        temp = self.start
        while temp.next != None:
            temp = temp.next
```

Salicid™

CamScanner

temp ⇒ last_node

temp.next = new_node . (will be under def enqueue():)

start Queue Empty:

```
class Node:

class Queue:
    def __init__(self):
        self.start = None

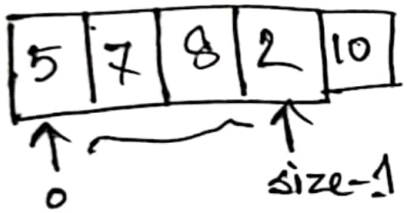
    def enqueue(self, elem):
        new_node = Node(elem, None)
        if self.start != None:
            temp = self.start
            while temp.next != None:
                temp = temp.next
            # temp ⇒ last_node
            temp.next = new_node
```

Array Based Implementation:

Linear Array

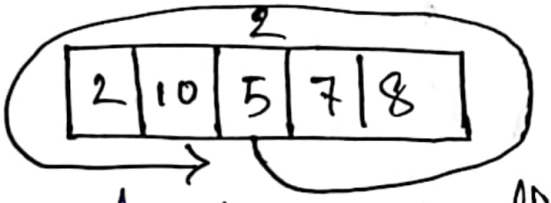
⇒ start = 0

⇒ last_idx = size - 1



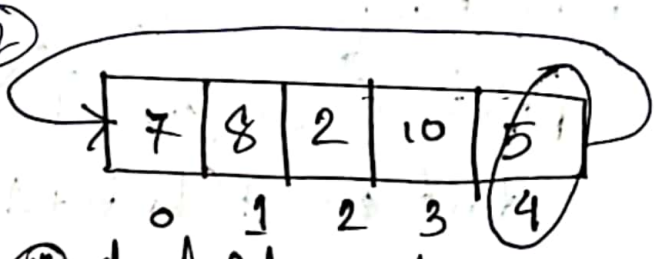
Circular Array (c_a)

①



✱ start_idx = 2 (in this case)

②



✱ start_idx = 4.

⇒ last_idx = (start + size - 1) % len(c_a)

for ① →

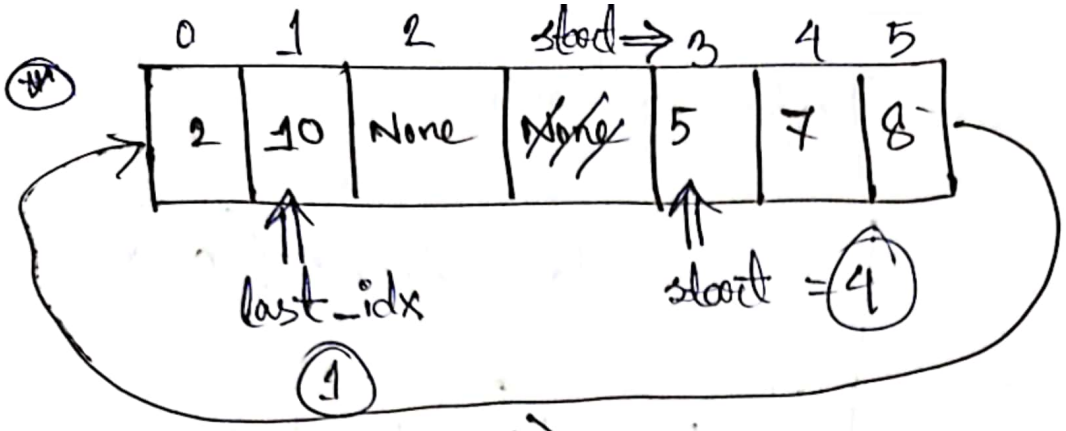
last_idx = (2 + 5 - 1) % 5

= 6 % 5 = 1

✱ n%5 = 0, 1, 2, 3, 4

Salicid™

CamScanner



Iteration (forward):

```
for i in range (size):
    print (c_a [start])
    start = (start + 1) % len (c_a)
```

OR

```
if start == len - 1:
    start = 0
```

Iteration (backward):

```
last_idx = (start + size - 1) % len (c_a)
for i in range (size):
    print (c_a [last_idx])
    last_idx -= 1
    if last_idx == -1:
        last_idx = len (c_a) - 1
```

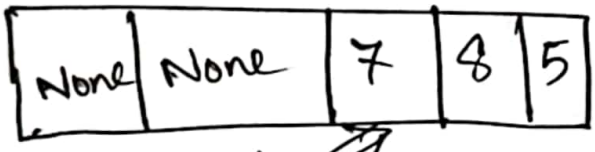
Queue Implementation:

Variation-01:

q = 

self.start

(Q with linear array)

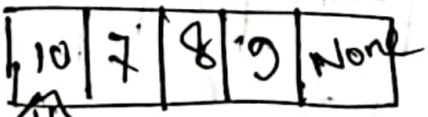
↳ 

self.start

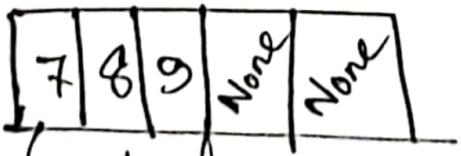
(✗) memory loss

Variation-02:

self.start = 0 → Always.

q = 

start

↳ q = 

start

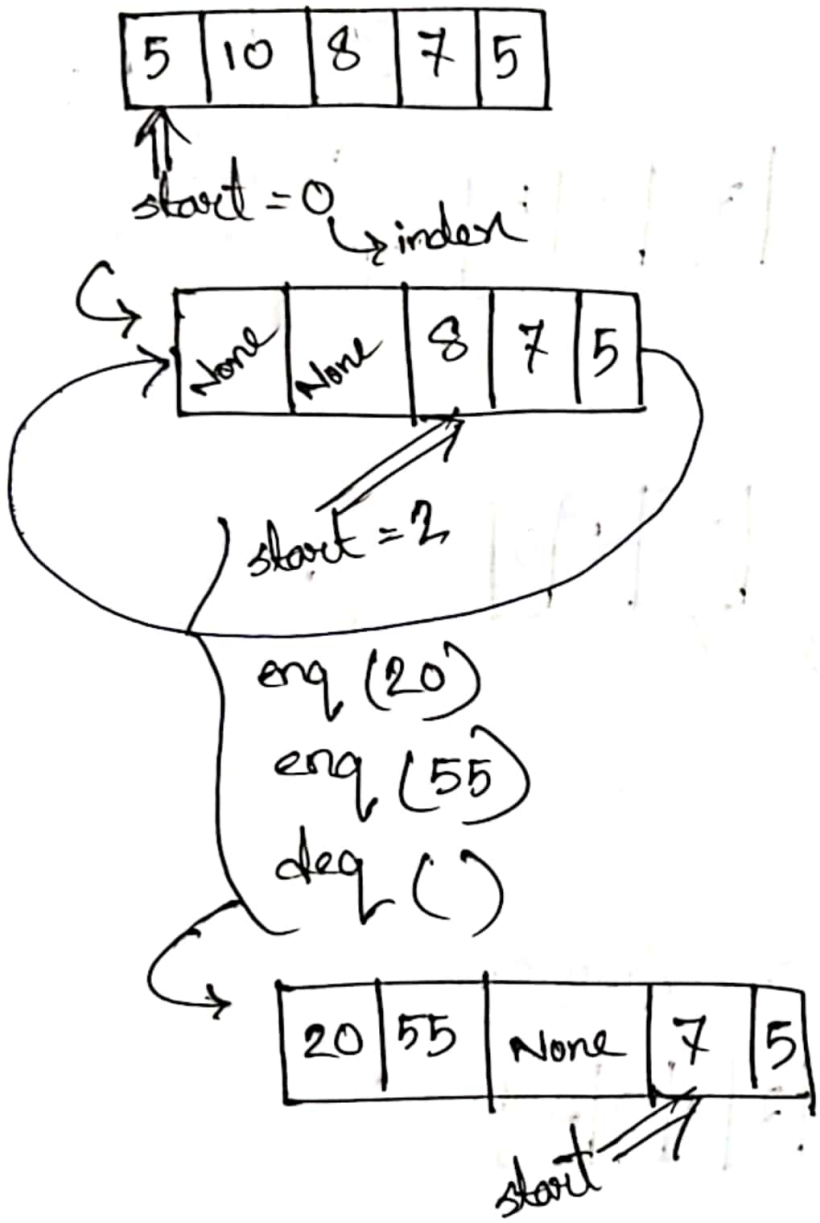
(✓) memory maintain!

(✗) time consuming!

Salicid™

CamScanner

Variation – 03:



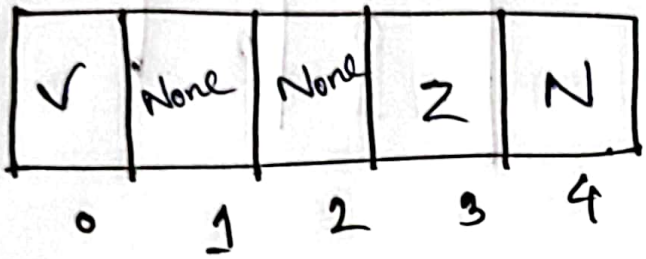
Circular Array

ZNV##PQ ##R?

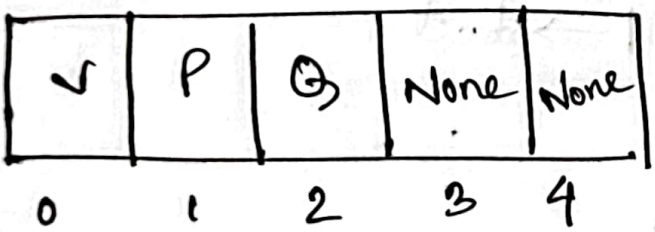
start : 3
length : 5

maintain it !

start	last
3	
4	
0	
1	



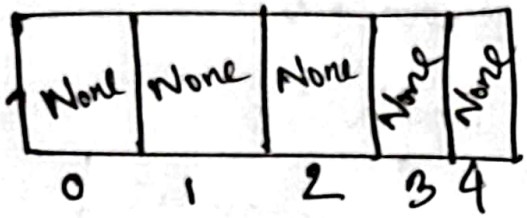
start : ~~3~~ 4



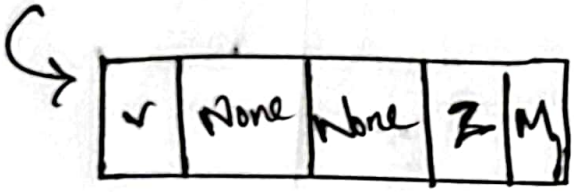


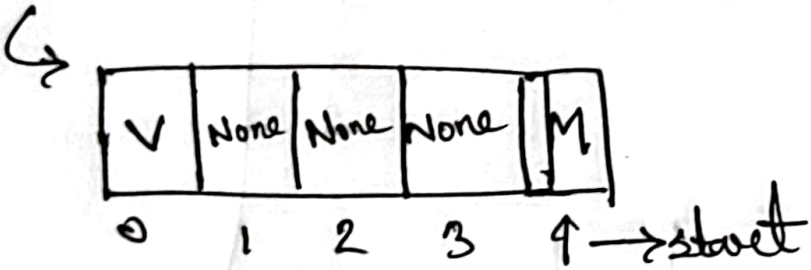
Salicid™

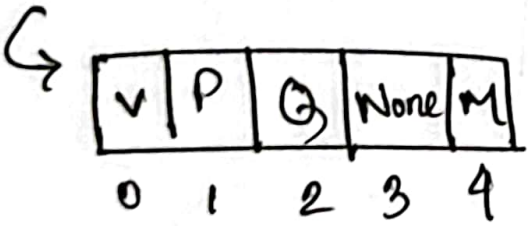
CamScanner

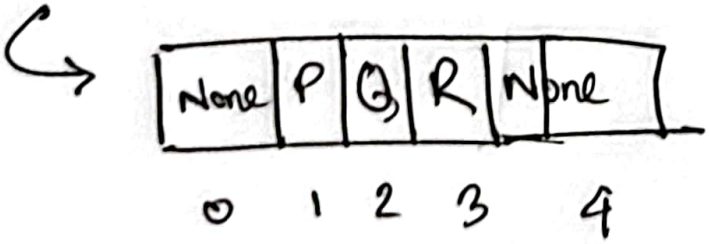


start	last	Queue
~~3~~	~~0~~	ZMV
~~4~~	~~1~~	
~~0~~	~~2~~	
1	3	









Quiz → Use Case + Simulation

CamScanner